

Lab_Assignment_9.2

AI Assisted Coding B37 2303A52077

Task 1 – Documentation – Function Summary Generation

Task:

Use AI to generate concise functional summaries for each Python function in a given script.

- Provide a Python script to the AI.
- Ask the AI to write a short summary describing the purpose of each function.
- Ensure summaries are brief and technically accurate.
- Do not include code implementation details

Code:

```
def factorial(n):  
    if n < 0:  
        return "Factorial is not defined for  
negative numbers."  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n-1)  
  
def main():  
    n = int(input("Enter a number: "))
```

```

        print("Factorial of", n, "is", factorial(n))
if __name__ == "__main__":
    main()

```

Prompt:

#Generate a proper Python docstring for each function in the below code.

AI Generated docstring with output:

```

Welcome  ninth.py x  BLACKBOX
ninth.py > ...
1  #factorial.py
2  #Generate a proper Python docstring for each function in the below code.
3  def factorial(n):
4      """
5      Docstring for factorial
6
7      :param n: Description
8      :type n: int
9      :return: Description of return value
10     :rtype: int
11
12     This function calculates the factorial of a given non-negative integer n.
13     The factorial of a non-negative integer n is the product of all positive integers less than or equal to n.
14     For example:
15     factorial(5) will return 120, since 5! = 5 * 4 * 3 * 2 * 1 = 120.
16     factorial(0) will return 1, since 0! is defined to be 1.
17     If n is negative, the function will return a message indicating that factorial is not defined for negative numbers.
18     """
19     if n < 0:
20         return "Factorial is not defined for negative numbers."
21     if n == 0:
22         return 1
23     else:
24         return n * factorial(n-1)
25 def main():
26     """
27     Docstring for main
28
29     This function serves as the entry point of the program.
30     It prompts the user to enter a number, calculates its factorial using the factorial function, and prints the result.
31     """
32     n = int(input("Enter a number: "))
33     print("Factorial of", n, "is", factorial(n))
34 if __name__ == "__main__":
35     main()

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

ted coding/ai_assi/ninth.py
Enter a number: 5
Factorial of 5 is 120
PS D:\Subjects\ai assisted coding\ai_assi>

```

Python Python Python

```
def factorial(n):
```

```
    """
```

Docstring for factorial

```
:param n: Description
:type n: int
:return: Description of return value
:rtype: int
```

This function calculates the factorial of a given non-negative integer n .

The factorial of a non-negative integer n is the product of all positive integers less than or equal to n .

For example:

`factorial(5)` will return 120, since $5! = 5 * 4 * 3 * 2 * 1 = 120$.

`factorial(0)` will return 1, since $0!$ is defined to be 1.

If n is negative, the function will return a message indicating that factorial is not defined for negative numbers.

```
...

if n < 0:
    return "Factorial is not defined for
negative numbers."

if n == 0:
    return 1

else:
```

```

        return n * factorial(n-1)
def main():
    '''
    Docstring for main

    This function serves as the entry point of the
    program.

    It prompts the user to enter a number,
    calculates its factorial using the factorial
    function, and prints the result.
    '''
    n = int(input("Enter a number: "))
    print("Factorial of", n, "is", factorial(n))
if __name__ == "__main__":
    main()

```

Explanation:

The script contains a function called `factorial(n)` that calculates the factorial of a number using recursion. It works for non-negative integers, and if the input is 0, the function returns 1. If a negative number is provided, the function returns an error message instead of performing the calculation.

Another function, `main()`, is responsible for interacting with the user. It asks the user to enter an integer, sends that value to the `factorial()` function, and then prints the result on the screen.

The script runs the `main()` function only when the file is executed directly, which is handled by the condition `if __name__ == "__main__":`. Additionally, docstrings are included above each

function to describe what the function does, what input it expects, and what output it returns. Overall, the program demonstrates a simple recursive approach to calculating factorials through a small command-line interaction.

Task -2 (Documentation – Logical Explanation for Conditions and Loops)

Task:

Use AI to document the logic behind conditional statements and loops in a Python program. Provide a Python program without comments.

- Instruct AI to explain only decision-making logic and loop behavior.
- Skip basic syntax explanations.

Code:

```
rows = int(input("Enter the number of rows for the
star pattern: "))
# Outer loop to iterate through each row
for i in range(1, rows + 1):
    # Inner loop to print stars for the current row
    for j in range(i):
        print("*", end="") # Print star without
moving to the next line
    print() # Move to the next line after printing
all stars for the current row
```

Prompt:

#starpattern.py

#Generate Python code with clear explanations describing the logic of conditions and loops.

AI Generated result & code output:

```
9.2.2.py > ...
1  #starpattern.py
2  #Generate Python code with clear explanations describing the logic of conditions and loops.
3
4  # This program generates a star pattern based on user input for the number of rows.
5  # The user is prompted to enter the number of rows for the star pattern.
6  # The program uses a nested loop to create the pattern. The outer loop iterates through each row,
7  # while the inner loop prints the appropriate number of stars for each row.
8  # Get the number of rows for the star pattern from the user
9
10 rows = int(input("Enter the number of rows for the star pattern: "))
11 # Outer loop to iterate through each row
12 for i in range(1, rows + 1):
13     # Inner loop to print stars for the current row
14     for j in range(i):
15         print("*", end="") # Print star without moving to the next line
16     print() # Move to the next line after printing all stars for the current row
17 # The outer loop starts from 1 and goes up to the number of rows specified by the user.
18 # The inner loop runs 'i' times for each iteration of the outer loop, which means
19 # it prints one more star than the previous row, creating a right-angled triangle pattern of stars.
20
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python +

```
PS D:\Subjects\ai assisted coding\ai_assi_codes\9.2> & C:/Users/ASUS/AppData/Local/Programs/Python/Python314/python
assited coding/ai_assi_codes/9.2/9.2.2.py"
Enter the number of rows for the star pattern: 6
*
**
***
****
*****
*****
PS D:\Subjects\ai assisted coding\ai_assi_codes\9.2> |
```

Explanation:

The code generated from a prompt instructing the creation of Python code for a star pattern program, emphasizing clear explanations of loop and condition logic. The prompt specified prompting the user for the number of rows and using nested loops to build a right-angled triangle of stars. The outer loop iterates from 1 to the user-input rows, while the inner loop prints 'i' stars per row (where 'i' is the current row

number), incrementally forming the pattern. This approach demonstrates basic input handling, loop nesting, and incremental printing without newlines until each row completes. The generated code includes detailed comments explaining each step, making it educational for understanding Python's control structures. Overall, the prompt-driven generation focuses on simplicity, readability, and logical clarity for beginners.

Task -3 (Documentation – File-Level Overview)

Task:

Use AI to generate a high-level overview describing the functionality of an entire Python file.

- Provide the complete Python file to AI.
- Ask AI to write a brief overview summarizing the file's purpose and functionality.
- Place the overview at the top of the file.

Prompt:

#Generate a entire Python file overview.

AI Generated overview:

```
9.2.1.py > ...
1  #Generate a entire Python file overview.
2
3  # This Python file contains two main components: a star pattern generator and a factorial calculator.
4  # The star pattern generator prompts the user to enter the number of rows and then uses nested loops
5  # to print a right-angled triangle pattern of stars.
6  # The factorial calculator defines a function to compute the factorial of a non-negative integer
7  # using recursion. It also includes a main function that serves as the entry point of the program,
8  # allowing the user to input a number and see its factorial result. Each function is accompanied by a detailed
9  # docstring explaining its purpose, parameters, return value, and examples of usage.
10
```

Task Description -4 (Documentation – Refine Existing Documentation)

Task:

Use AI to improve clarity and consistency of existing documentation in Python code.

- Provide Python code containing basic or unclear comments.
- Ask AI to rewrite the documentation to improve clarity and consistency.
- Ensure technical meaning remains unchanged.

Prompt 1:

#generate a simple Python code containing unclear comments.

AI Generated code:

```
# This program does something with numbers.
```

```
def do_something(n):
```

```
    # Check if n is less than 10
```

```
    if n < 10:
```

```
        # If it is, do this
```

```
        return n * 2
```

```
    else:
```

```
        # Otherwise, do that
```

```
        return n + 5
```

```
# Get a number from the user
```

```
number = int(input("Enter a number: "))
```

```
# Call the function and print the result
```

```
result = do_something(number)
```

```
print("The result is:", result)
```

Prompt 2:

Update Python code with refined and improved documentation that is clear and consistent.

update Python code with refined and improved documentation that is clear and consistent.

```
# This program does something with numbers.
def do_something(n):
    """# Check if n is less than 10
    ... if n < 10:
    ..... # If it is, do this
    ..... return n * 2
    ... else:
    ..... # Otherwise, do that
    ..... return n + 5
# Get a number from the user
number = int(input("Enter a number: "))

# Call the function and print the result
result = do_something(number)
print("The result is:", result)
```

Modify selected code

```
# This program does something with numbers.
def do_something(n):
    # Check if n is less than 10
    """
    ... Doubles the input if less than 10, otherwise adds 5.
    ...
    ... Args:
    ...     n (int): The input number to process
    ...
    ... Returns:
    ...     int: The processed result
    ... """
    if n < 10:
        # If it is, do this
        return n * 2
    else:
        # Otherwise, do that
        return n + 5
# Get a number from the user
# Get user input
number = int(input("Enter a number: "))
# Call the function and print the result
# Process the number and display the result
result = do_something(number)
print("The result is:", result)
```

Final Code & its Output:

```

2
3 def do_something(n):
4     """
5     Doubles the input if less than 10, otherwise adds 5.
6
7     Args:
8         n (int): The input number to process
9
10    Returns:
11        int: The processed result
12    """
13    if n < 10:
14        return n * 2
15    else:
16        return n + 5
17
18    # Get user input
19    number = int(input("Enter a number: "))
20
21    # Process the number and display the result
22    result = do_something(number)
23    print("The result is:", result)
24
25

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

Enter a number: 5
The result is: 10
PS D:\Subjects\ai assisted coding\ai_assi_codes\9.2>

```

Explanation:

This code defines a function that processes a number based on its value and includes user input and output. The comments are clear and explain the purpose of each part of the code.

Whereas previous code was inconsistent with unclear comments

Task -5 (Documentation – Prompt Detail Impact Study)

Task:

Study the impact of prompt detail on AI-generated documentation quality.

Create two prompts: one brief and one detailed.

- Use both prompts to document the same Python function.
- Compare the generated outputs.

Prompt 1 (brief):

#Write documentation for a Python function that calculates the factorial of a number using recursion. Include a short description of what the function does.

AI Generated code:

```

3  def factorial(n):
4      """
5          Calculates the factorial of a given non-negative integer n using recursion.
6
7          The factorial of a non-negative integer n is the product of all positive integers less than or equal to n.
8          For example:
9          factorial(5) will return 120, since 5! = 5 * 4 * 3 * 2 * 1 = 120.
10         factorial(0) will return 1, since 0! is defined to be 1.
11         If n is negative, the function will return a message indicating that factorial is not defined for negative n
12
13         Args:
14             n (int): A non-negative integer for which to calculate the factorial.
15
16         Returns:
17             int or str: The factorial of n if n is non-negative, or an error message if n is negative.
18         """
19         if n < 0:
20             return "Factorial is not defined for negative numbers."
21         if n == 0:
22             return 1
23         else:
24             return n * factorial(n-1)
25
26     # Example usage:
27     print(factorial(5)) # Output: 120
28     print(factorial(0)) # Output: 1
29     print(factorial(-3)) # Output: Factorial is not defined for negative numbers.
30

```

Prompt 2 (detailed):

"""

Act as a Python documentation assistant. Write detailed documentation for a Python function that calculates the factorial of a number using recursion.

Include:

- A clear description of the function's purpose.
- Explanation of how recursion is used.
- Description of the input parameter and expected data type.
- Description of the return value.
- Possible edge cases (such as input 0 or negative numbers).

- Example usage of the function.
- Well-structured Python docstrings following standard documentation practices.

'''

AI Generated Code:

```

9.2.5.1.py > ...
1  '''
2  Act as a Python documentation assistant. Write detailed documentation for a Python function that calculates the factorial
3  of a number using recursion.
4
5  Include:
6  - A clear description of the function's purpose.
7  - Explanation of how recursion is used.
8  - Description of the input parameter and expected data type.
9  - Description of the return value.
10 - Possible edge cases (such as input 0 or negative numbers).
11 - Example usage of the function.
12 - Well-structured Python docstrings following standard documentation practices.
13 '''
14
15 def factorial(n):
16     """
17     Calculates the factorial of a given non-negative integer n using recursion.
18
19     The factorial of a non-negative integer n is the product of all positive integers less than or equal to n.
20     For example:
21     factorial(5) will return 120, since 5! = 5 * 4 * 3 * 2 * 1 = 120.
22     factorial(0) will return 1, since 0! is defined to be 1.
23     If n is negative, the function will return a message indicating that factorial is not defined for negative numbers.
24
25     Args:
26         n (int): A non-negative integer for which to calculate the factorial.
27
28     Returns:
29         int or str: The factorial of n if n is non-negative, or an error message if n is negative.
30     """
31     if n < 0:
32         return "Factorial is not defined for negative numbers."
33     if n == 0:
34         return 1
35     else:
36         return n * factorial(n-1)
37
38 # Example usage:
39 print(factorial(5)) # Output: 120
40 print(factorial(0)) # Output: 1
41 print(factorial(-3)) # Output: Factorial is not defined for negative numbers.

```

Comparsion table:

Aspect	Brief Prompt Documentation	Detailed Prompt Documentation
Completeness	Provides a short description of the factorial function and basic explanation. Some elements like recursion explanation or edge cases may be minimal.	More comprehensive documentation including purpose, recursion explanation, parameters, return values, edge cases, and examples.
Clarity	Clear but concise explanation. Suitable for quick understanding but may lack deeper explanation for beginners.	Very clear and structured with sections such as description, parameters, return values, and examples, making it easier to understand.
Accuracy	Correctly explains factorial calculation and recursion but with limited detail.	Accurate explanation with additional context on recursion and behavior for different inputs.
Structure	Basic docstring with limited formatting.	Well-structured documentation with organized sections following standard

Aspect	Brief Prompt Documentation	Detailed Prompt Documentation
Edge Case Handling	Mentions negative numbers briefly.	documentation practices. Explicitly describes edge cases like $n = 0$ and negative numbers.
Example Usage	May include simple examples.	Includes clear example usage demonstrating different inputs and outputs.
Maintainability	Useful but may require additional documentation later.	Better for long-term maintenance because the documentation is more detailed and descriptive.